

Algoritmo Genético aplicado à resolução do problema das N-rainhas

Caio Cardoso Dos Santos	11202021632
Joel da Silva Junior	11201920366
Victor Ravazio de Lima	11201920941

Computação Evolutiva e Conexão
Professor: Mateus Coelho Silva

1 - Introdução

O problema das N-rainhas (N-Queens) é um problema combinatório clássico muito estudado nas áreas de ciência da computação e matemática. Ele consiste em posicionar N rainhas em um tabuleiro de xadrez de dimensão $N \times N$ de forma que nenhuma delas possa atacar a outra, respeitando as regras de movimentação da peça no jogo de xadrez [1].

Para que uma solução seja válida, devem ser atendidas três restrições: não podem existir mais de uma rainha na mesma linha, na mesma coluna ou na mesma diagonal (principal ou secundária), caracterizando-o como um problema de satisfação de restrições, que busca encontrar configurações que atendam simultaneamente a todas as restrições impostas [1].

Além de seu valor teórico, este problema possui diversas aplicações práticas e teóricas, sendo amplamente utilizado na área de inteligência artificial, problemas de otimização, teoria dos jogos e etc [1]. Nesse contexto, o presente trabalho busca oferecer uma estratégia de solução baseada em algoritmos genéticos, explorando mecanismos evolutivos para encontrar configurações válidas de forma eficiente.

2 - Referencial Teórico

2.1 - Algoritmos Genéticos

Algoritmos Genéticos são métodos de **otimização** e **busca** inspirados nos mecanismos de **evolução de populações** de seres vivos. Esses algoritmos seguem o princípio da seleção natural e sobrevivência do mais apto, declarado em 1859 pelo naturalista e fisiologista inglês Charles Darwin em seu livro “A Origem das Espécies” [2].

Otimização é a busca da **melhor solução** para um dado problema. Consiste em tentar várias soluções e utilizar a informação obtida neste processo de forma a encontrar soluções cada vez melhores [2].

As técnicas de busca e otimização, geralmente, apresentam:

- Um **espaço de busca (S)**, onde estão todas as possíveis soluções do problema;
- Uma **função objetivo (f)**, que é utilizada para avaliar as soluções produzidas, associando a cada uma delas uma nota (**Fitness**);
- Uma **Paisagem de Fitness**, que é a topologia criada por **f** sobre **S**.

Em termos matemáticos, a otimização consiste em achar a solução que corresponda ao ponto de **máximo** ou **mínimo** da função objetivo. Quando esse ponto é o maior/menor valor que a função pode atingir, ele é chamado de máximo/mínimo **global**, sendo esse o ponto de melhor solução para a função, e quando esse ponto é maior/menor que a vizinhança dele, ele é chamado de máximo/mínimo **local** [2].

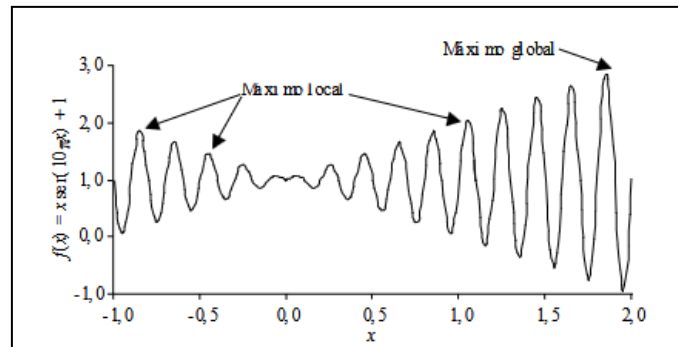


Figura 1 - Gráfico da função $f(x) = x \sin(10\pi x) + 1$.

2.2 - Etapas de um algoritmo genético

O primeiro passo de um Algoritmo Genético típico é a geração de uma **população inicial** de indivíduos, que é formada por um conjunto aleatório de indivíduos que representam possíveis soluções do problema a ser resolvido [2].

Durante o **processo evolutivo**, esta população é **avaliada** e cada cromossomo recebe uma **nota de aptidão** (fitness), refletindo a qualidade da solução que ele representa [2]. Como será feito esse cálculo de aptidão depende do problema sendo analisado.

Em geral, os indivíduos mais aptos são **selecionados** e os menos aptos são descartados [2].

Os membros selecionados podem sofrer modificações em suas características fundamentais através dos operadores de **crossover** e **mutação**, gerando descendentes para a próxima geração [2].

Este processo é repetido até que uma solução satisfatória seja encontrada. Quando se conhece uma resposta máxima/mínima da função objetivo, pode-se utilizar esse valor como critério de parada. O algoritmo 2.2 ilustra bem este procedimento [2].

Algoritmo 2.2: Algoritmo Genético típico

Seja $S(t)$ população de indivíduos na geração t .

$t \leftarrow 0$

inicializar a população $S(t)$

enquanto o critério de parada não for satisfeito faça

$t \leftarrow t + 1$

avaliar $S(t)$ (cálculo de fitness)

selecionar $S(t)$ a partir de $S(t-1)$

aplicar **crossover** sobre $S(t)$

aplicar **mutação** sobre $S(t)$
 avaliar $S(t)$
fim enquanto

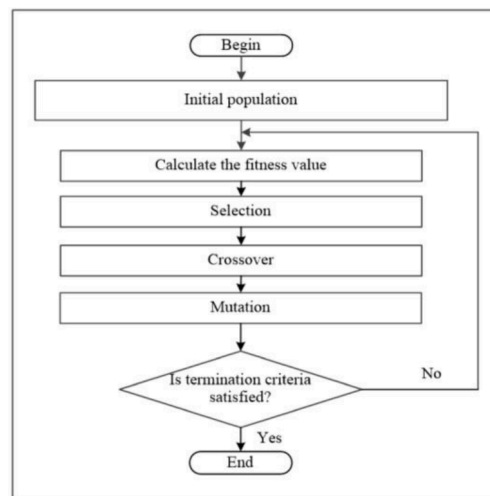


Figura 2 - Fluxograma das etapas de um algoritmo genético.

As seções seguintes descrevem com mais detalhes cada etapa desse algoritmo.

2.2.1 - Mecanismos de seleção

Os mecanismos de seleção são estratégias para o algoritmo genético eliminar soluções a cada época. A seleção é feita com base no nível de aptidão do indivíduo (fitness), e o quão rigorosa será essa seleção pode ser ajustado sob as necessidades do problema, sendo denominada como **taxa de seleção**. As estratégias de seleção podem ser feitas de diferentes maneiras, apresentadas a seguir.

2.2.1.1 - Seleção por torneio

Nessa estratégia, **n** indivíduos aleatórios são escolhidos (com probabilidades iguais) e o indivíduo com melhor aptidão (fitness) é selecionado para a população intermediária. O processo repete-se até preencher a população intermediária. Utiliza-se, geralmente, o valor **n=3**.

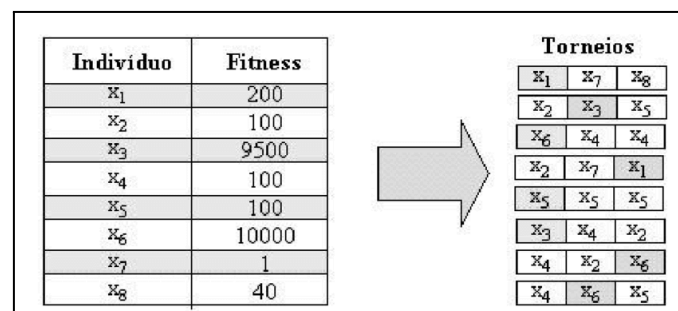


Figura 3 - Seleção por torneio.

Esse método resulta em uma **maior diversidade** da população intermediária, ou seja, das soluções exploradas e consideradas, por conta do seu elemento aleatório e a seleção ocorrer em pequenos “blocos”. Porém, isso faz com que a **convergência** para a melhor solução seja **mais lenta**.

2.2.1.2 - Seleção por roleta

Nessa estratégia, se seleciona os sobreviventes ou eliminados baseado na probabilidade proporcional à aptidão do indivíduo (fitness) [2]. Portanto, a probabilidade de seleção p_i de um indivíduo s_i com aptidão f_i é dada por:

$$p_i = \frac{f_i}{\sum_{i=1}^N f_i}$$

Fórmula 1 - Probabilidade de seleção.

Calcula-se uma coluna de aptidões acumuladas. Em seguida, gera-se um número aleatório r (tirado de uma distribuição uniforme) no intervalo $[0, \text{SOMATOTAL}]$, onde SOMATOTAL é a soma de todas as aptidões. Por fim, o indivíduo selecionado é o primeiro (segundo a tabela de cima para baixo) que possui aptidão acumulada maior que r . Os mesmos passos são repetidos até preencher a população intermediária [2].

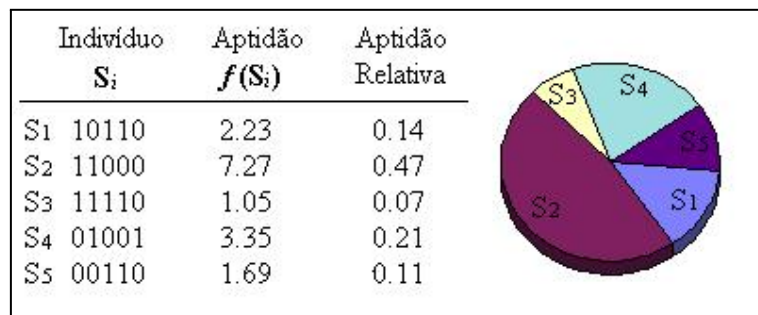


Figura 4 - Seleção por roleta.

Esse método resulta em uma **menor diversidade** da população intermediária, por estarmos comparando todos os indivíduos ao mesmo tempo e os selecionando com base em um fator em comum. Porém, a **convergência** para a melhor solução ocorre de forma **mais rápida** quando comparado ao método de seleção por torneio.

2.2.1.3 - Seleção por elitismo

Nessa estratégia, rankeia-se os indivíduos por aptidão (fitness), e selecionamos os indivíduos mais aptos, ou seja, mais bem ranqueados nessa ordenação, determinados por uma linha de corte a partir do número de indivíduos que queremos na população intermediária.

Esse método apresenta uma **diversidade ainda menor** e **convergência mais rápida**, pelos mesmos motivos de estarmos realizando uma comparação entre todos os indivíduos sob o mesmo fator (fitness), sendo mais rápido por ter apenas uma etapa de seleção.

2.2.2 - Mecanismos de crossover

Os mecanismos de crossover representam as estratégias utilizadas para a etapa de geração de novos indivíduos com base naqueles selecionados na etapa anterior. Pares aleatórios são formados a partir da população intermediária, e parte das informações de cada um dos indivíduos pareados é reproduzida em um ou mais novos indivíduos construídos. Isso introduz variabilidade de soluções em nossa população, a partir da reprodução de características selecionadas com boa aptidão para o problema. Em alguns problemas, pode-se existir uma **taxa de crossover**, indicando a probabilidade do par formado formar novos indivíduos com a informação cruzada dos dois, porém nosso problema das N-rainhas não usa esse tipo de taxa. A forma como o crossover é feito depende da representação do problema.

1. Representação por Permutação

São os problemas em que as soluções são representadas por diferentes **permutações**, ou seja, uma determinada ordem ou posição em que os diferentes dados são postos. O nosso problema das N-rainhas é um problema de representação por permutação!

Nesse tipo de representação nós temos o mecanismo de crossover chamado **order crossover**, no qual se seleciona uma subseção de um dos indivíduos pareados, indivíduo este escolhido de forma aleatória, e os elementos ordenados presentes nesta subseção são reproduzidos no novo indivíduo na mesma posição, os demais elementos são selecionados de forma ordenada do outro indivíduo pareado, desde que esses elementos não sejam os mesmos desta subseção selecionada. Esse método pode ser modificado para ter múltiplas seções e diferentes tamanhos de subseção, com o objetivo de passar mais ou menos informações de um dos indivíduos do par.

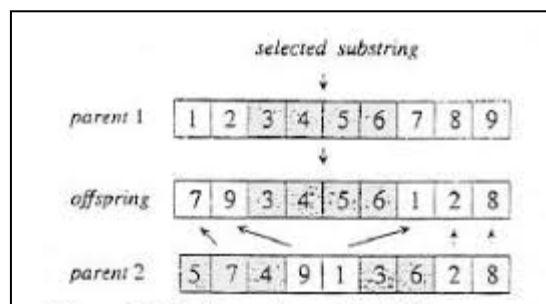


Figura 5 - Order crossover.

2. Representação Binária

São problemas em que as soluções são representadas pela presença ou não de determinado elemento, ou seja, uma representação binária de 'presente' (1) ou 'não presente' (0).

Nesse tipo de representação, nós temos três estratégias de crossover, sendo elas:

- **Crossover de um ponto:** Seleccionamos um ponto de corte no vetor de elementos e reproduzimos os elementos de um dos indivíduos do par antes desse ponto, e do outro indivíduo após o ponto.

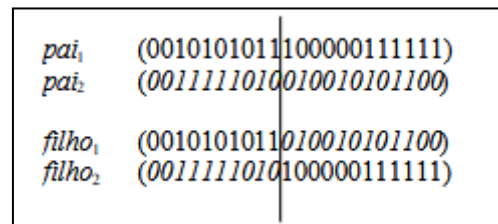


Figura 6 - Crossover de um ponto.

- **Crossover de múltiplos pontos:** Mesma estratégia do anterior, mas com seleção de múltiplos pontos, intercalando entre a reprodução dos elementos dos diferentes indivíduos do par.

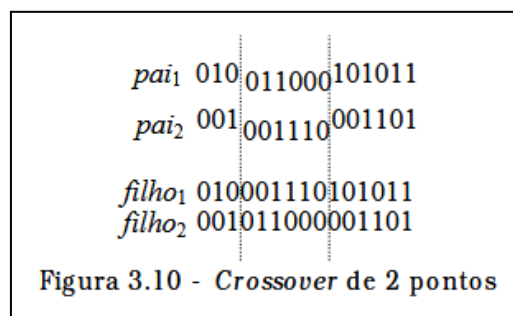


Figura 7 - Crossover de múltiplos ponto.

- **Crossover uniforme:** Cada bit é escolhido de qualquer um dos pais com igual probabilidade. Certas vezes, outras proporções de misturas são usadas, resultando em descendentes que herdam mais informações de um dos indivíduos do par.

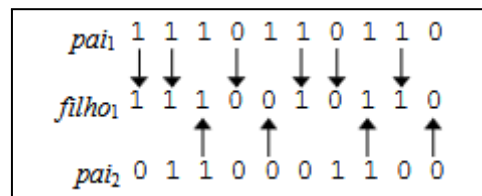


Figura 8 - Crossover uniforme.

3. Representação Real

São problemas em que as soluções são representadas por números reais.

Nesse tipo de representação, nós temos duas estratégias de crossover, sendo elas:

- **Média:** O descendente adota o valor médio entre os dois indivíduos do par formado.
- **Blend:** Gera o novo indivíduo em um intervalo expandido ao redor dos indivíduos do par, controlado pelo parâmetro α .

$$c = p_1 + \beta(p_2 - p_1)$$

onde $\beta \in U(-\alpha, 1 + \alpha)$.

Fórmula 2 - Fórmula de Blend.

Outras estratégias podem ser usadas para esse tipo de representação (média geométrica, linear, aritmético, simples) [2], mas fogem da ementa deste curso.

2.2.3 - Mecanismos de mutação

Os mecanismos de mutação ditam a mudança nas soluções representadas nos novos indivíduos gerados no crossover. Tem como objetivo introduzir mais variabilidade nas soluções da nossa população, evitando uma convergência prematura. Ela também ocorre de acordo com uma **taxa de mutação**.

1. **Representação por permutação:** Fazemos uma **mudança de ordem** (order swap), em que se escolhe duas posições aleatórias e se troca os valores entre elas.
2. **Representação Binária:** Fazemos um **bit flip**, ou seja, a troca do valor de cada bit de acordo com a taxa de mutação.
3. **Representação Real:** Fazemos mutação por **distribuição**, ou seja, substituímos parte do indivíduo selecionado para mutação por um número extraído de uma distribuição, seja ela uniforme ou não uniforme.

2.3 - O problema das N-Rainhas

O Problema das N-rainhas surgiu inicialmente restrito ao subcaso das 8-rainhas pelo enxadrista alemão Max Bezzel em 1848 e, generalizado para N-rainhas pelo também enxadrista francês 1869 [3]. Ele consiste em dispor n rainhas num tabuleiro de xadrez de ordem $N \times N$ de forma que nenhuma das peças ataquem entre si. Para isso, nenhuma das rainhas pode estar na mesma linha, coluna ou diagonal (principal ou secundária). A seguir há uma representação de ataque entre duas rainhas nessas três situações.

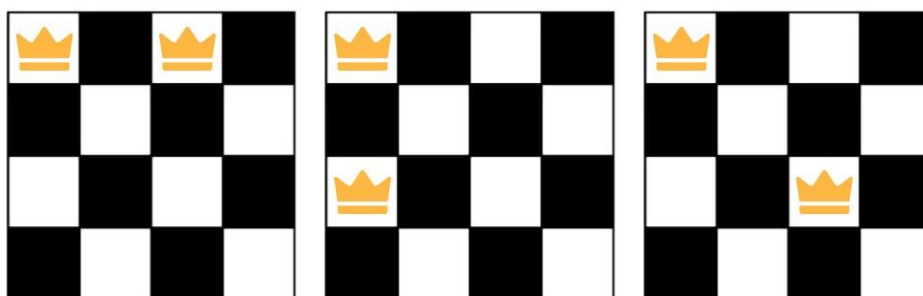


Figura 9 - Rainhas se atacando pela linha, coluna ou diagonal.

O problema das N-rainhas pode ser classificado como do tipo NP-difícil. Neles, não se conhece atualmente algoritmos que o resolvam em tempo de complexidade polinomial. Uma maneira de se resolver esse problema é de forma ingênua por força-bruta, em que testa-se todas as possibilidades do espaço de busca S à procura das soluções válidas. No entanto, esse método possui complexidade de tempo exponencial $O(N^n)$, inviabilizando sua utilização já para pequenos valores de N .

Outra forma de se resolvê-lo é por meio da aplicação do algoritmo *Backtracking*, em que a inserção de uma nova rainha no tabuleiro depende que ela não ataque as já inseridas. Caso seja apontada uma das três situações de ataque, volta-se (*back*) ao caso da inserção da rainha anterior na linha acima, a fim de movê-la à direita até que não haja ataque às rainhas existentes. Caso se chegue ao fim das colunas possíveis à direita, volta-se à rainha anterior na linha acima e assim sucessivamente. Encerrado os ataques volta-se a priorizar a inserção da última rainha que provocou a etapa *back*, prosseguindo para as próximas linhas.

A implementação clássica do *Backtracking* realiza esse processo de forma recursiva até que todas as n rainhas sejam posicionadas no tabuleiro. Isso permite reduzir a busca de soluções válidas em S , atingindo complexidade de tempo fatorial $O(n!)$. Porém, novamente conforme a dimensão do tabuleiro N aumenta, torna-se custoso sua execução.

Uma forma de representar a busca de soluções das N -rainhas se dá pela árvore de recursão a seguir. Nela, há os casos em que até aquele nível da árvore não haviam ataques entre si. Na ilustração há a árvore de recursão para o problema das 4-rainhas [3]. Para esse caso, somente 2 soluções são válidas, ilustradas pelo contorno vermelho dos nós folha centrais.

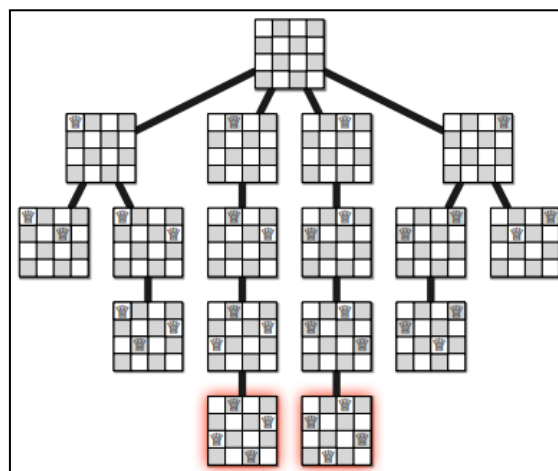


Figura 10 - Árvore de recursão para o problema das 4-rainhas

O problema original das 8-rainhas possui 92 soluções válidas possíveis. Segue a seguir uma possível solução.

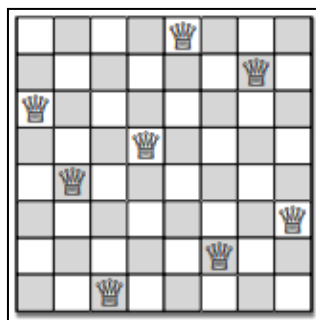


Figura 11 - Possível solução para o problema das 8-rainhas.

3 - Metodologia

3.1 - Modelagem do Problema

Como explicado em seções anteriores, o problema das N-rainhas consiste em posicionar n rainhas em um tabuleiro de xadrez de dimensão $n \times n$, de modo que nenhuma ataque a outra. Dessa forma, para modelar o problema, cada solução candidata (indivíduos da população) é representada como uma permutação dos números de 0 a $n-1$, onde:

- O índice do vetor representa a coluna do tabuleiro
- O valor armazenado nesta posição representa a linha onde a rainha está posicionada
- n é o tamanho do tabuleiro

Esse tipo de modelagem garante automaticamente a satisfabilidade duas das três restrições do problema:

1. Não existem duas rainhas na mesma linha (pois a permutação não possui valores repetidos)
2. Não existem duas rainhas na mesma coluna (cada índice do vetor representa uma coluna distinta)

Assim, resta verificar apenas a restrição de que não existem duas rainhas na mesma diagonal. Essa tarefa é aquela na qual está embasado a modelagem do fitness adotada para o algoritmo genético desenvolvido.

A função de fitness adotada para a avaliação do quão boa é uma solução, discutida mais detalhadamente na seção 3.3, consiste no cálculo do número de conflitos diagonais entre pares de rainhas. O objetivo é minimizar esse valor, sendo portanto, uma solução ótima, aquela com zero conflitos.

Vale registrar que, para esse problema, não é necessário o uso de um dataset. Isso ocorre devido a natureza combinatória e o fato dele possuir múltiplas soluções possíveis para uma mesma dimensão de tabuleiro. Dessa forma, ao invés de buscar a convergência do algoritmo genético para uma solução previamente conhecida, o algoritmo gera soluções candidatas (indivíduos da população) e avalia diretamente sua qualidade via função de fitness, podendo convergir para qualquer uma das soluções válidas, sem depender de um conjunto pré-definido de respostas.

3.2 - Inicialização da população

A instância de entrada do problema é definida pelo parâmetro n , que corresponde à dimensão do tabuleiro $n \times n$ e, consequentemente, ao número de rainhas a serem posicionadas. A partir desse valor, constrói-se uma população inicial gerada de forma aleatória, composta por um conjunto de indivíduos (soluções candidatas), cada um representado por uma permutação de inteiros de 0 a $n-1$.

O tamanho da população é definido pelo hiperparâmetro popsize, enquanto a aleatoriedade na sua geração tem como intuito promover diversidade entre os indivíduos e oferecer uma exploração mais ampla do espaço de busca nas etapas subsequentes do algoritmo.

3.3 - A função de Fitness

A função de fitness é responsável por avaliar a qualidade de cada solução candidata, contabilizando o número de conflitos diagonais entre as rainhas. A partir da representação apresentada na seção anterior, onde $P[a] = b$, (isto é, a rainha está na coluna a, linha b), duas rainhas nas posições $(i, P[i])$ e $(j, P[j])$, estão na mesma diagonal se:

$$\bullet \quad |i - j| = |P[i] - P[j]| \quad (1)$$

A condição (1) é verificada para cada par de índices $i < j$, e caso ela seja satisfeita, um conflito é contabilizado. Ao final, o valor de fitness no fim corresponde ao número total de conflitos diagonais do indivíduo (permutação) avaliado. Essa abordagem caracteriza o problema como um problema de minimização, onde soluções com menor número de conflitos são melhores, sendo considerada ótima aquela cujo fitness é igual a zero.

3.4 - Ciclo evolutivo

O ciclo evolutivo do algoritmo genético é executado ao longo de um número máximo de épocas previamente estabelecido em num_epochs. Além desse limite, adota-se também um critério de parada alternativo: uma condicional que verifica se a solução ótima foi encontrada (fitness igual a zero). Para cada época, são realizadas as seguintes etapas:

1. Cálculo de Fitness:

Cada indivíduo da população é avaliado por meio da função de fitness que contabiliza o número de colisões. Quanto menor o número de colisões, melhor a solução.

2. Seleção:

Utiliza-se o mecanismo de seleção por **torneio**, onde pares de indivíduos são escolhidos aleatoriamente e aquele com pior fitness (maior número de conflitos) é eliminado. Esse processo se repete até que tamanho da população atinja o valor correspondente ao produto popsize x survival_rate.

3. Reprodução (crossover):

Para repopular a população, aplica-se o mecanismo de reprodução **order crossover**:

- um ponto de corte é selecionado aleatoriamente
- o descendente herda a parte inicial de um dos pais
- os genes restantes são preenchidos na ordem em que aparecem no segundo pai, evitando repetir aqueles já advindos do pai 1

Esta etapa é concluída quando o tamanho da população atinge o valor estabelecido no hiperparâmetro popsize.

4. Mutação:

Após o crossover, cada descendente pode sofrer uma variabilidade em seus genitores em função de uma probabilidade definida no hiperparâmetro `mutation_prob`. O mecanismo utilizado é o **swap mutation**, que consiste em:

- Selecionar duas posições aleatórias do vetor
- Trocar os valores entre essas posições

5. Critério de parada:

Como antecipado, o processo evolutivo é encerrado quando uma solução ótima é encontrada ou quando o número máximo de épocas é atingido

Ao final, espera-se que a população tenha convergido para indivíduos com baixo número de conflitos, idealmente em uma solução ótima para o problema.

4 - Resultados e Discussão:

O algoritmo genético foi avaliado em diferentes dimensões do problema das N-rainhas, utilizando como instâncias de entrada os valores $N = 4, 5, 8, 10, 16, 32, 50, 100, 150$ e 200 . Visando garantir relevância estatística, foram realizadas 10 execuções independentes para instâncias menores ($N < 50$) e 5 execuções para instâncias médias ($N \geq 50$ e $N < 70$). Para instâncias maiores, foi considerada apenas uma execução devido ao elevado tempo de execução. A tabela a seguir resume o desempenho do algoritmo desenvolvido sob as circunstâncias descritas:

Tamanho do tabuleiro (N)	Número de execuções	S	Emed	Emin	Tempo de execução médio (s)	Tempo de execução mínimo (s)	Média do tempo de uso de CPU (s)	Média do uso de RAM final (MB)	Média do pico de uso de RAM (MB)
4	10	10	1	1	0.00182	0.00100	0.00156	148.67	148.67
5	10	10	1	1	0.00266	0.00199	0.00313	157.82	157.82
8	10	10	1	1	0.02576	0.00499	0.02656	153.04	153.04
10	10	10	6	1	0.40289	0.00901	0.40313	153.56	153.56
16	10	9	325	16	34.15018	1.19779	33.96719	160.14	160.14
32	10	7	390	38	96.30872	6.43995	95.45625	151.83	151.83
50	5	3	545	107	247.68146	49.73968	247.01250	131.07	146.03

100	1	0	1000	1000	1749.79 65	1749. 7965	1744.10 94	91.81	98.35
150	1	0	1000	1000	3322.14 452	3322. 14452	3280.48 437	27.87	154.04
200	1	0	1000	1000	6138.11 31	6138. 1131	6128.32 81	55.29	93.60

Onde:

- |S| - Número de soluções ótimas encontradas
- |Emed| - Número médio de épocas para achar a solução ótima (se encontrou)
- |Emin| - Número mínimo de épocas para achar a solução ótima (se encontrou)

A partir da tabela, observa-se que para instâncias pequenas ($N = 4, 5, 8$), o algoritmo encontrou soluções ótimas de forma imediata, conforme indicado pelos valores:

- Emed = 1
- Emin = 1

Esse comportamento indica que a solução ótima já estava presente na população inicial, e assim não houve a necessidade de execução efetiva do ciclo evolutivo, afinal após o cálculo do fitness dos indivíduos, o primeiro critério de parada foi satisfeito já na primeira época. Para fins didáticos, a figura a seguir apresenta gráficos de convergência da população para $N = 4, 5$ e 8 , desconsiderando a primeira condição de parada e executando sobre todo o número de épocas.

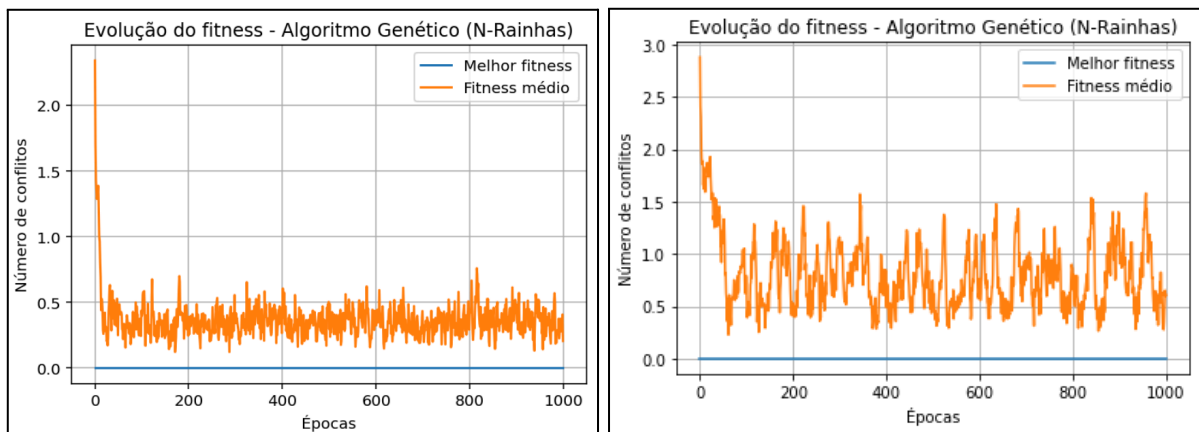


Figura 12 - Convergência para tabuleiro tamanho 4, 5.

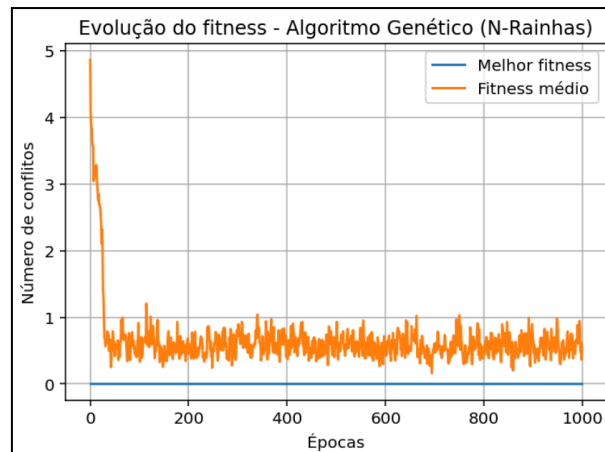


Figura 13 - Convergência para tabuleiro tamanho 8.

Na figura acima fica notável o processo de convergência da população ao longo das épocas. Observa-se que uma solução ótima foi encontrada na primeira época (fitness = 0), entretanto o fitness médio da população ainda estava elevando. Conforme as épocas foram passando, o fitness médio foi decrescendo até se estabilizar em valores próximos de 0. O comportamento oscilatório (serrilhado) da curva de fitness médio pode ser explicado pelo uso do mecanismo de seleção de torneio, onde estamos optando pela diversidade, o que por consequência permite a sobrevivência de indivíduos menos “capazes” quando pareados com outros indivíduos menos capazes. Além disso, outros hiperparâmetros como a taxa de sobrevivência e mutação também influenciam na geração destas pequenas flutuações.

À medida que o tamanho do problema aumenta, observa-se um crescimento no número médio de épocas necessárias para encontrar soluções ótimas.

Para $N = 10$, temos:

- Emed = 6
- Emin = 0

Nesse caso, em média, o algoritmo precisa de mais gerações para convergir, embora na melhor execução ele convergiu muito rapidamente (na primeira época). Segue na figura o gráfico de convergência de uma das execuções da instância de tamanho $N = 10$

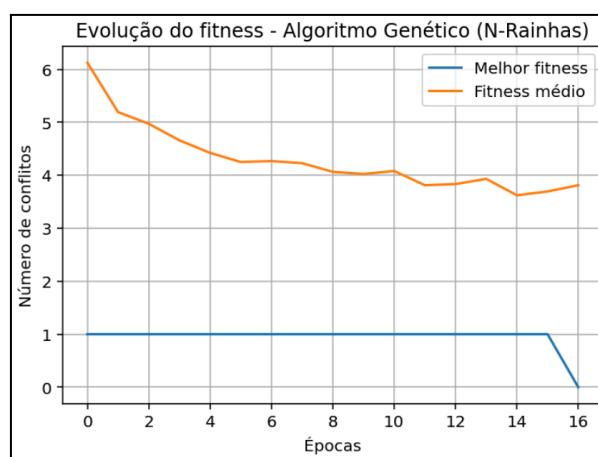


Figura 14 - Convergência da população em uma das execuções para $N = 10$.

Podemos observar na figura acima que na primeira geração existe uma solução muito próxima da que desejávamos, com fitness = 1. A solução ótima, porém, foi atingida somente na 17ª época. Por outro lado, o fitness médio da população começou elevado (em torno de 6), mas foi caindo ao longo das épocas, aproximando-se de uma solução ótima, embora não dê tempo de se estabilizar próximo de zero. Isso indica que embora um indivíduo possa encontrar uma solução ótima de forma precoce, são necessárias mais épocas para a população convergir totalmente para este valor.

Para instâncias de tamanho médio e grande, observamos dois comportamentos principais. Primeiramente, ocorre um aumento expressivo no número de épocas médias e mínimas necessárias para encontrar soluções ótimas. Além disso, é notável que nem todas as execuções conseguem encontrar uma solução ótima, ou seja, atingir o fitness = 0. Empiricamente obtemos o seguinte:

- Para $N = 16$: 9 soluções ótimas encontradas em 10 execuções
- Para $N = 32$: 7 soluções ótimas encontradas em 10 execuções
- Para $N = 50$: 3 soluções ótimas encontradas em 5 execuções
- Para $N \geq 100$: Nenhuma solução ótima encontrada

Esses resultados indicam que o algoritmo apresenta uma baixa escalabilidade sob os parâmetros adotados (que estão mantidos fixos). Apesar disso, os gráficos de convergência para estas instâncias mostram que a população continua evoluindo ao longo das épocas e ela se aproxima, mesmo que lentamente, da solução ótima, apesar de não atingi-la.

Uma explicação possível para este comportamento está relacionada ao crescimento do espaço de busca, afinal o número de permutações possíveis de N elementos sem repetição é $N!$. Com isso, uma população pequena para um valor de N grande, combinado com um mecanismo de seleção que torna a convergência mais lenta, e uma taxa de sobrevivência muito baixa, certamente contribuem para este desempenho observado. As figuras a seguir ilustram a convergência das instâncias de tamanhos maiores.

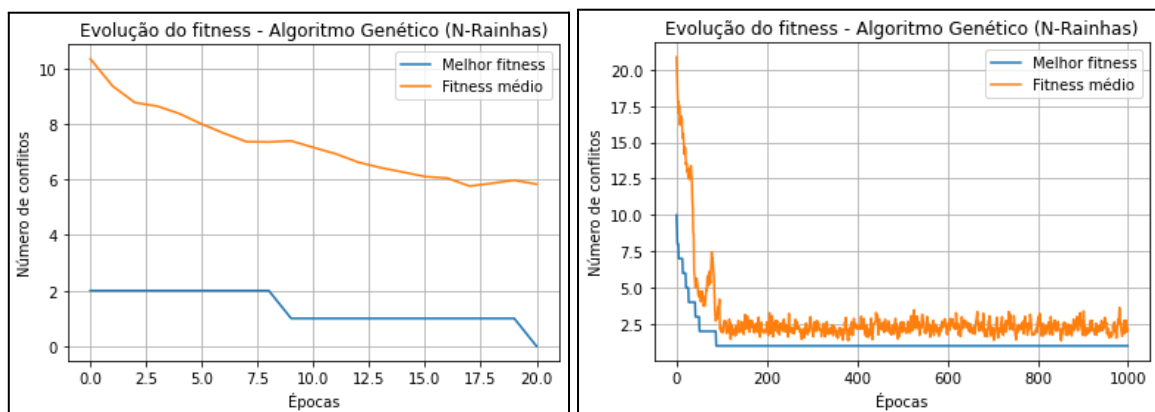


Figura 15 - Convergência da população em uma das execuções para $N = 16$ e 32 .

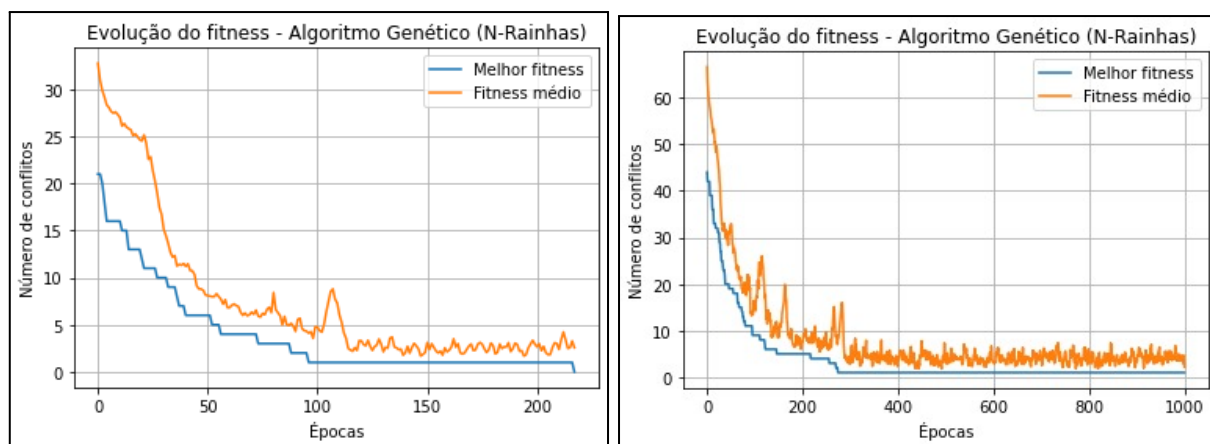


Figura 16 - Convergência da população em uma das execuções para $N = 50$ e 100 .

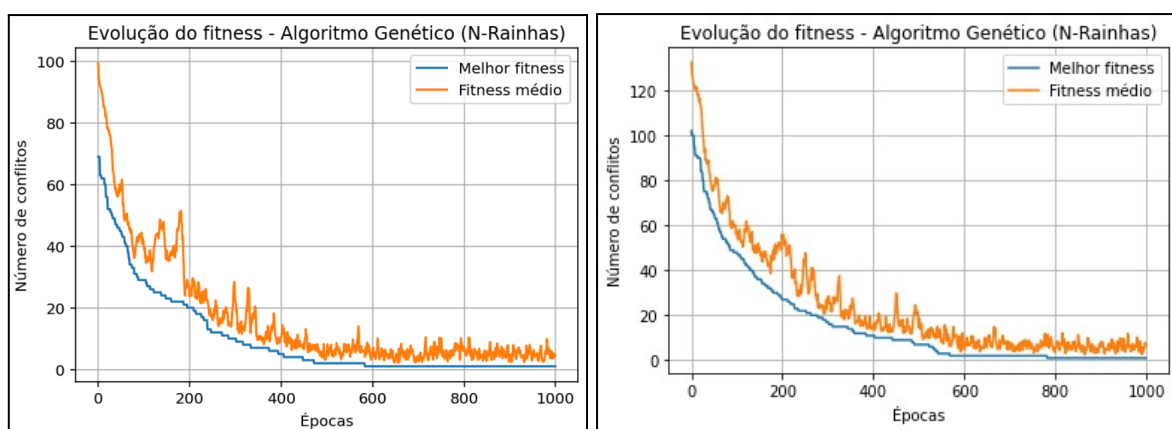


Figura 17 - Convergência da população nas execuções com $N = 150$ e 200 .

5 - Conclusão:

A partir dos resultados obtidos, podemos concluir que o algoritmo genético desenvolvido apresentou um desempenho satisfatório. Para pequenos casos, o algoritmo encontra soluções ótimas de imediato (na primeira época), sem a necessidade de execução do processo evolutivo. Conforme o tamanho da instância do tabuleiro aumenta, o espaço de busca S cresce fatorialmente. Entretanto, o processo evolutivo implementado induz a convergência da população para indivíduos cujos valores de fitness estejam cada vez mais próximos do ótimo global do problema.

Além disso, é importante considerar a aplicação de diferentes métodos de seleção, a fim de explorar estratégias distintas que se apresentem mais adequadas à natureza do problema e favoreçam uma convergência mais rápida. Vale reforçar, por fim, que também é válido a reavaliação de hiperparâmetros visando explorar de forma mais eficiente o espaço de busca S e melhorar a capacidade de otimização e generalização.

6 - Declaração de ética (IA Generativa):

Declaramos total autoria e responsabilidade pelo conteúdo do projeto aqui presente, com ferramentas de IA generativa sendo utilizadas apenas para tradução de artigos em inglês para melhor compreensão. Nenhum uso foi feito na realização tanto do código, quanto do conteúdo do relatório. O código foi baseado no material prático desenvolvido em sala de aula.

Referências:

- [1] KAVYA. N-Queens Problem: An Overview. Medium, 14 jan. 2025. Disponível em: <https://medium.com/@kavya1234/n-queens-problem-an-overview-e7b2b3cc0bd5>. Acesso em: 27 abr. 2026.
- [2] DE LACERDA, Estéfane GM; DE CARVALHO, ACPLF. Introdução aos algoritmos genéticos. Sistemas inteligentes: aplicações a recursos hídricos e ciências ambientais, v. 1, p. 99-148, 1999. Disponível em: <https://www.academia.edu/download/48970437/ag.pdf>. Acesso em: 28 abr. 2026.
- [3] ERICKSON, Jeff. **Algorithms**. [S.l], 2018. Disponível em: <https://jeffe.cs.illinois.edu/teaching/algorithms/book/02-backtracking.pdf>. Acesso em: 28 abr. 2026.